

openagent-logger — DATASHEET

Cross-service contract reference. The README is for newcomers; this file is for engineers integrating with openagent-logger or auditing what it owns.

Field	Value
Service name	openagent-logger
Version	0.1.0
Role	Capture layer for the OpenAgent system
Port	8003
Base image	python:3.11-slim
Runtime user	openagent (uid 1000)
Database	PostgreSQL 13+, shared instance
Schema	openagent_logger
Connectivity	HTTP (inbound); PostgreSQL TCP (outbound)
Status	working — pre-production

1. Ownership boundaries

What this service owns

Domain	Concrete artifact
Event ingestion endpoint	POST /events
Operational telemetry storage	openagent_logger.ops_events
Conversation capture storage	openagent_logger.conversation_captures
Security-audit storage	openagent_logger.audit_events
Partition lifecycle	src/partitioning.py, src/scheduler.py
HMAC verification + storage	src/security.py, hmac_signature column
Replay-window enforcement	LOGGER_REPLAY_WINDOW_SECONDS
The schema definition	database/init.sql
The retention policy	LOGGER_RETENTION_*_DAYS env vars

Domain	Concrete artifact
The shared network + Postgres setup	<code>scripts/setup-network.*</code> , the documented Postgres bring-up

What this service does NOT own

Concern	Owner	Notes
Session lifecycle	Not implemented	<code>session_id</code> accepted but never validated
User identity	Not implemented	<code>user_id</code> column nullable, always null today
PII stripping	Not implemented	Captures stored raw
Rate limiting	Reverse proxy	Defer to upstream
Backup / restore	DB platform (Render, RDS, etc.)	Standard Postgres backup
Encryption at rest	DB platform	Same
Encryption in transit	Platform / TLS terminator	HTTP between trusted services
Authentication of users	openagent-api	openagent-logger only authenticates services

This list is the canonical answer to "should openagent-logger do X?" — if X is in the right-hand column, no.

2. Inbound contracts

All endpoints expect `Content-Type: application/json` on requests with a body. All endpoints return `application/json`.

The error envelope is the same on every endpoint:

```
{
  "success": false,
  "error": "HTTP_401",
  "message": "Invalid or missing API key",
  "timestamp": "2026-05-14T18:24:01.342000+00:00"
}
```

2.1 POST /events

Capture a signed event.

Headers

Header	Value	Required
X-API-Key	<LOGGER_API_KEY>	yes
Content-Type	application/json	yes

Body — common envelope (every event type)

Field	Type	Required	Notes
event_type	enum string	yes	ops_event conversation_capture audit_event
request_id	UUID	yes	Correlation ID for the originating /chat call
source_service	string ≤64	yes	Emitter identifier (e.g. openagent-api)
client_timestamp	ISO-8601 datetime	yes	Emitter wall-clock; replay-window-checked
session_id	string ≤64 null	no	Reserved correlation field; nullable, never validated; null OK
user_id	UUID null	no	Reserved correlation field; nullable, null today
hmac_signature	64-char hex	yes	HMAC-SHA256 over the canonical string
payload	object	yes	Per-event_type shape, see below

session_id and user_id are accepted-but-never-validated fields. The logger stores whatever a caller supplies (or null). They exist so a caller that *does* track sessions or users can correlate events without a schema change. In the reference stack, openagent-api emits both as null.

Payload — when event_type = ops_event

Field	Type	Required	Notes
action	string ≤128	yes	e.g. request_received
outcome	string ≤32	yes	e.g. success, failure, timeout
details	object	no	Free-form structured context

Payload — when event_type = conversation_capture

Field	Type	Required	Notes
input_text	string 1..200_000	yes	Raw user input
output_text	string 0..1_000_000	yes	Visible model answer
input_hash	64-char hex	yes	SHA-256 of input_text
output_hash	64-char hex	yes	SHA-256 of output_text

Field	Type	Required	Notes
<code>model_used</code>	string ≤255	no	Model identifier from openagent-infra
<code>reasoning_effort</code>	enum string	no	<code>low</code> <code>medium</code> <code>high</code>
<code>latency_ms</code>	int ≥0	no	End-to-end /chat latency
<code>input_tokens</code>	int ≥0	no	Reported by model
<code>output_tokens</code>	int ≥0	no	Reported by model

Payload — when `event_type = audit_event`

Field	Type	Required	Notes
<code>actor</code>	string ≤128	yes	Who initiated the action
<code>action</code>	string ≤128	yes	What was attempted
<code>target</code>	string ≤255	no	What was acted on
<code>outcome</code>	string ≤32	yes	<code>success</code> , <code>failure</code> , <code>denied</code> , ...
<code>ip_address</code>	string ≤45	no	IPv4 or IPv6, stored as PostgreSQL <code>INET</code>
<code>details</code>	object	no	Additional context

Canonical string (for `hmac_signature`)

```
{request_id}|{client_timestamp_iso}|{event_type}|
{sha256(canonical_payload_json)}
```

where `canonical_payload_json` is:

```
json.dumps(payload, sort_keys=True, separators=(",", ":"),
            default=str, ensure_ascii=False)
```

HMAC-SHA256 over the canonical string, keyed with `LOGGER_HMAC_SECRET.encode("utf-8")`. Result is the lowercase 64-char hex digest.

Success — 201 Created

```
{
  "success": true,
  "event_id": "f17c84d7-3b9e-4a02-9a45-7f5e9d2c1abc",
  "event_type": "ops_event",
  "message": "Event captured",
}
```

```
{
  "timestamp": "2026-05-14T18:24:01.512000+00:00"
}
```

Error cases

Status	Cause
401 Unauthorized	Missing/invalid <code>X-API-Key</code> , or HMAC signature mismatch, or replay window exceeded
400 Bad Request	Discriminator mismatch (payload doesn't match <code>event_type</code>), or Pydantic validation failure
422 Unprocessable Entity	Pydantic field-level validation (hex format, length bounds)
500 Internal Server Error	DB persistence failure
503 Service Unavailable	DB connection probe failed at startup; service reports degraded

2.2 GET /health

Report service readiness.

Headers

Header	Value	Required
<code>X-API-Key</code>	<code><LOGGER_API_KEY></code>	yes

Success — 200 OK

```
{
  "status": "ok",
  "service": "openagent-logger",
  "version": "0.1.0",
  "database": "connected",
  "scheduler": "running",
  "timestamp": "2026-05-14T18:24:01.000000+00:00"
}
```

`status` is the worst of (`database`, `scheduler`):

Combination	status
DB connected + scheduler running	ok
DB connected + scheduler stopped	degraded

Combination	status
DB disconnected (any)	unhealthy

Use **status** for orchestrator decisions, not the nested fields. The in-container HEALTHCHECK in **docker/logger/Dockerfile** parses this field specifically — Docker's view of "healthy" matches the service's self-reported state, not just HTTP 200.

2.3 GET /stats

Return row counts and timestamp bounds across all three tables.

Headers

Header	Value	Required
X-API-Key	<LOGGER_API_KEY>	yes

Success — 200 OK

```
{
  "ops_events_count": 12453,
  "conversation_captures_count": 871,
  "audit_events_count": 14,
  "oldest_event": "2026-04-15T00:01:23.456000+00:00",
  "newest_event": "2026-05-14T18:23:55.012000+00:00",
  "timestamp": "2026-05-14T18:24:01.000000+00:00"
}
```

For partitioned tables Postgres satisfies **COUNT(*)** via parallel sequential scans across partitions. This is **not** cheap on large tables; treat **/stats** as an admin endpoint, not a hot path.

2.4 GET /

Unauthenticated service identification.

Success — 200 OK

```
{
  "service": "openagent-logger",
  "version": "0.1.0"
}
```

Exposes only what is already visible in HTTP response headers. Useful for orchestrators that cannot pre-share **X-API-Key** for a healthcheck.

3. Outbound contract

3.1 Database

`openagent-logger` has exactly **one** outbound dependency: the shared PostgreSQL instance. `openagent-logger` owns the bring-up of both the shared Postgres and the `openagent-network` Docker network it lives on (see the README's setup section). The connection contract below is what other components rely on.

Role provisioning (init time)

The `openagent_logger` role is **fully provisioned at init time** by `database/init.sql`. There is no separate post-init `ALTER ROLE` step.

The mechanism:

1. The Postgres container is started with
`-e PGOPTIONS="-c logger.db_password=${LOGGER_DB_PASSWORD}"`.
2. `PGOPTIONS` is read by `libpq` when `psql` connects to the server during `docker-entrypoint-initdb.d/` processing.
3. The server applies `-c logger.db_password=...` as a session-level custom GUC.
4. `init.sql` reads the GUC via `current_setting('logger.db_password')` and uses the value to `CREATE ROLE openagent_logger ... PASSWORD '...'` (or `ALTER ROLE ... WITH PASSWORD '...'` if the role already exists, so re-running `init.sql` keeps the role and `.env` in sync).
5. If the GUC is missing or empty, `init.sql` aborts with a clear error rather than creating an unauthenticatable role.

The flow ensures `.env`'s `LOGGER_DB_PASSWORD` is the single source of truth for the role's password. The `openagent-logger` service reads the same `.env` value at startup for its connection string, so the two halves are always aligned by construction.

Connection

- Connection URL: `LOGGER_DATABASE_URL` (preferred) OR assembled from `LOGGER_DB_USER`, `LOGGER_DB_PASSWORD`, `LOGGER_DB_HOST`, `LOGGER_DB_PORT`, `LOGGER_DB_NAME`.
- Pooling: SQLAlchemy default (`pool_size=5`, `max_overflow=10`, `pool_pre_ping=True`, `pool_recycle=300`).
- The `openagent_logger` DB role:
 - Owns the `openagent_logger` schema and all tables within it.
 - Has `SELECT` and `INSERT` granted (no `UPDATE`, no `DELETE`).
 - Has `DROP TABLE` on partitions via ownership, used at retention time.

- Has its password provisioned at init time from the `logger.db_password` GUC (see Role provisioning above) — not via a separate operator step.

Schema scoping

The logger lives in its own schema (`openagent_logger`) inside the shared `openagent_shared` database. Tables in `openagent_logger.*`:

Table	Retention	Partition strategy
<code>ops_events</code>	<code>LOGGER_RETENTION_OPS_DAYS</code> (default 90 days)	<code>RANGE (created_at)</code> monthly
<code>conversation_captures</code>	<code>LOGGER_RETENTION_CONVERSATION_DAYS</code> (default 180 days)	Same
<code>audit_events</code>	<code>LOGGER_RETENTION_AUDIT_DAYS</code> (default 2555 days / ~7 years)	Same

All three share a common envelope: `event_id`, `request_id`, `source_service`, `created_at`, `client_timestamp`, `session_id`, `user_id`, `hmac_signature`, `retention_class`. The composite primary key on every table is (`event_id`, `created_at`) — required by PostgreSQL because `created_at` is the partition key.

Schema-scoping (rather than a dedicated database) keeps the door open: another service could later share the same instance under its own schema and role without colliding with this one, and the logger can be lifted out to a dedicated Postgres at any time by changing only the connection URL. Today the instance hosts only `openagent_logger.*`.

Write semantics

- Append-only. There is no code path in this service that issues `UPDATE` or `DELETE` against event rows.
- Retention is implemented by `DROP TABLE <expired_partition>`, not row-level `DELETE`.
- Partition creation is idempotent (`CREATE TABLE IF NOT EXISTS ... PARTITION OF ...`).
- Each `POST /events` is one transaction containing one `INSERT`.

Read semantics

- `/stats` queries `COUNT(*)`, `MIN(created_at)`, `MAX(created_at)` on each table.
- `/health` issues `SELECT 1` against the connection pool.
- The service itself does not read event rows back; it only writes them.

3.2 No other outbound calls

`openagent-logger` does **not** make HTTP calls to any other service.

4. State model

4.1 In-memory state

The service holds three pieces of process-internal state:

State	Lifetime	Notes
SQLAlchemy engine + connection pool	Process lifetime	Initialized at module import
FastAPI <code>app</code> and routes	Process lifetime	Initialized at module import
<code>RetentionScheduler</code> (APScheduler)	Process lifetime	Started by lifespan, stopped on SIGTERM

There is **no other in-memory state**. Restarting the container has no data loss beyond the very small set of in-flight HTTP requests.

4.2 Durable state

100% of durable state lives in PostgreSQL, in the `openagent_logger` schema, as described in §3.1.

4.3 Cross-event state

There is none. Each event is processed independently. The `request_id` in the envelope correlates related events post-hoc via query, not via runtime state in the service.

5. Configuration

The full env-var reference lives in the README and the template lives in `.env.example`. This datasheet includes only the contract-relevant values.

Variable	Default	Contractual?
<code>LOGGER_PORT</code>	<code>8003</code>	Yes — emitters point at this
<code>LOGGER_API_KEY</code>	(required)	Yes — emitters need it
<code>LOGGER_HMAC_SECRET</code>	(required)	Yes — emitters must compute matching sigs
<code>LOGGER_REPLAY_WINDOW_SECONDS</code>	<code>300</code>	Yes — affects acceptance of <code>client_timestamp</code>

Variable	Default	Contractual?
LOGGER_DB_PASSWORD	(required)	Yes — also passed to Postgres via PGOPTIONS at init time (see §3.1)
LOGGER_DB_SCHEMA	openagent_logger	Yes — consumers read from here
LOGGER_DB_NAME	openagent_shared	Yes — database name
LOGGER_RETENTION_*_DAYS	varies	Reference only — consumers should not assume any specific value

6. Integration notes

6.1 openagent-api (the only emitter)

`openagent-api` is the **only** emitter today. The integration:

1. Client module — `openagent-api`: `src/client/logger.py`

Wraps event construction, HMAC computation, and the HTTP POST. The signing function is duplicated from `openagent-logger`: `src/security.py` until both stabilise and a shared library is extracted.

2. Fire-and-forget pattern

`logger.py` enqueues each event onto an in-process `asyncio.Queue` and returns immediately. A background task drains the queue and POSTs to `openagent-logger`. `openagent-api` never blocks `/chat` on a logger response.

3. Event timing

- `request_received` ops_event — emitted on `/chat` ingress, after auth passes.
- `upstream_call` ops_event — emitted before calling `openagent-infra`.
- `upstream_error` ops_event — emitted on any `openagent-infra` failure.
- `stream_complete` ops_event — emitted after the SSE stream closes.
- `conversation_capture` — emitted after `stream_complete`, with the full assembled `output_text`.

4. session_id / user_id

`openagent-api` emits both as `null`. There is no session or user tracking in the reference stack. The logger accepts and stores whatever is supplied (or null) without validation.

5. Auth chain

Header on inbound /chat

X-API-Key: OPENAGENT_API_KEY (from frontend)

Header on outbound /events

(not forwarded)

X-API-Key: LOGGER_API_KEY (set fresh, per-boundary)

The transport keys are **compartmentalized** — `openagent-api` uses `LOGGER_API_KEY` for its outbound call, never `OPENAGENT_API_KEY`. This is the same per-boundary pattern `openagent-api` uses to call `openagent-infra` with `INFRA_API_KEY`.

6.2 Offline HMAC re-verification (any consumer)

Any consumer reading rows out of `conversation_captures` (or any table) can independently re-verify integrity without trusting the storage layer:

1. Re-compute `canonical_payload_json` from the stored `payload` using the exact `json.dumps` kwargs in §2.1.
2. Re-build the canonical string `{request_id}|{client_timestamp}|{event_type}|{payload_hash}`.
3. Re-compute HMAC-SHA256 keyed with `LOGGER_HMAC_SECRET` and compare (constant-time) against the stored `hmac_signature` column.

A mismatch catches bit-rot in the storage layer, any bug that wrote a row with a wrong signature, or tampering between capture and read. This is the value of storing the signature on the row — verification survives the original transport call being long forgotten.

A read-only consumer should connect with its own DB role scoped to `SELECT` on only the tables it needs, e.g.:

```
CREATE ROLE my_reader WITH LOGIN PASSWORD '<from env>';
GRANT USAGE ON SCHEMA openagent_logger TO my_reader;
GRANT SELECT ON openagent_logger.conversation_captures TO my_reader;
```

The logger exposes no HTTP read API for stored events; reads go through the database directly.

7. Failure modes

7.1 Database connection lost

Behaviour: `pool_pre_ping` catches stale connections on next use. `pool_recycle=300` proactively reconnects every 5 minutes. New POST

events fail with **500**. `/health` reports **database: disconnected**, overall **unhealthy**.

Recovery: automatic when the DB returns. No service restart needed.

7.2 HMAC verification fails

Behaviour: `POST /events` returns **401**, the request is logged as **Event rejected (request_id=..., type=...): Invalid HMAC signature**. No row is written.

Common causes:

- Emitter and receiver out of sync on the canonical-string format.
- **LOGGER_HMAC_SECRET** mismatch between sides.
- Field-order or escaping inconsistency in payload serialization.

7.3 Replay window exceeded

Behaviour: `POST /events` returns **401** with **message: client_timestamp outside replay window (skew Ns > 300s)**. No row is written.

Common causes:

- Emitter clock badly skewed (NTP failure).
- Event sat in an emitter-side queue too long.
- Replay of a captured event.

Mitigation: investigate clock sync first; raise **LOGGER_REPLAY_WINDOW_SECONDS** only as a last resort.

7.4 Partition missing for write time

Should not happen. The retention scheduler creates next-month partitions every day at **LOGGER_RETENTION_SCHEDULE_HOUR**, and **database/init.sql** seeds the first two months at install time. If it does happen (catastrophic clock skew, scheduler stopped):

Behaviour: `POST /events` returns **500**, log shows **no partition of relation "<table>" found for row**.

Recovery: hit `/health` to confirm scheduler is stopped, restart the service. Lifespan startup runs `_ensure_partitions()` immediately before registering the cron job.

7.5 Scheduler stopped (but DB up)

Behaviour: `/health` returns **degraded**. `POST` events still succeed (writes do not depend on the scheduler). Retention does not run until restart.

Risk: if the service stays up but the scheduler is dead for a long time, partitions for future months are not created. Eventually a write will land outside any partition. See §7.4.

7.6 Disk pressure on Postgres

`openagent-logger` has no way to throttle itself based on DB disk pressure.

At full disk, `INSERT`s fail with `500` and `/health` reports

`unhealthy` (the `SELECT 1` probe also fails once WAL cannot flush).

Mitigation: monitor Postgres disk separately; tighten retention windows if growth outpaces capacity.

7.7 `logger.db_password` GUC missing at init time

Behaviour: `init.sql` aborts with `ERROR: Required GUC 'logger.db_password' is not set.` and an actionable hint pointing at

the `PGOPTIONS` flag on `docker run`. No role, no schema, no tables are created. The Postgres container exits because the entrypoint init script failed.

Common causes:

- Operator forgot the `-e PGOPTIONS="-c logger.db_password=..."` flag.
- `LOGGER_DB_PASSWORD` was empty in `.env` when the shell read it.

Recovery: `docker rm -f openagent-shared-db`, fix the `docker run` command per the README, re-run.

8. Operational characteristics

These are **expectations**, not guarantees. Measure in your own deployment.

Property	Expected value
p50 latency, <code>POST /events</code>	< 20ms (small payloads)
p99 latency, <code>POST /events</code>	< 100ms
Throughput (single instance)	~500–1000 events/sec
Image size	~180 MB
Memory at idle	~80 MB
Memory under load	< 200 MB
Startup time (image already pulled)	~2–3 seconds

`conversation_capture` payloads can be large (up to ~1 MB `output_text`).
Latency for those scales with payload size.

9. Version history

Version	Notes
0.1.0	Initial release. Capture layer: <code>POST /events</code> , three partitioned tables (<code>ops_events</code> , <code>conversation_captures</code> , <code>audit_events</code>), HMAC verification + replay window, in-process retention scheduler. Pre-production.

10. Cross-references

- `README.md` — primary documentation, design rationale, security model
 - `src/api.py` — FastAPI app and routes
 - `src/schemas.py` — Pydantic schemas (source of truth for §2)
 - `src/models.py` — SQLAlchemy models (source of truth for §3.1)
 - `src/security.py` — HMAC + replay-window logic
 - `database/init.sql` — DDL (source of truth for the schema)
 - `.env.example` — every env var the code reads
-

openagent-logger — part of the OpenAgent system